Guía de desarrollo — Wilson

Wilson David Hernández Sánchez · M3 Solicitudes · M5 Mensajería · Offero, versión del 27 de octubre de 2026

> Esta guía es tu hoja de ruta para tus módulos: qué construir, en qué orden, con qué tablas y permisos, y cómo saber que quedó bien. No trae código: el código lo escribes tú. Las historias con puntos van en el Product Backlog del plan (Parte 5). Si algo aquí choca con el documento de contexto, manda el documento y le avisas a Diego. Lo marcado con [SUPUESTO] es una propuesta que todavía hay que confirmar.

Tu resumen

Módulos	M3 Solicitudes (complejidad media, ≈28 h) y M5 Mensajería (media-alta: ≈24 h completo, ≈14 h mínimo).
Carga estimada	≈42 h con M5 mínimo, frente a ≈47 h netas: cabe. M5 completo (tiempo real) solo si sobra tiempo y el PO lo aprueba.
Tu ventaja	Tu app de tareas en Flask es muy parecida a M3: crear, listar, ver el detalle y cambiar el estado.
Tu prioridad de la primera semana	Git en equipo. El código va en archivos con historial de commits, nunca en un .rar.
Desbloqueas a	Diego (requests para M4) e Iker (moderar solicitudes y reglas de reseñas).
Dependes de	Daniel (M0 y categories, mar 10 nov) y Diego (M1; quotes para M5, mar 17 nov).
Fechas clave	requests y formulario dom 15 nov · M3 terminado dom 22 nov · enviar y ver mensajes dom 29 nov · M5 terminado dom 6 dic.

De Flask a Next.js: lo que ya sabes, con otro nombre

En Flask	En Next.js (Offero)
`@app.route("/tareas")`	Carpeta `src/app/solicitudes/` con su `page.js`
`/tareas/`	Carpeta `[requestId]`
Plantillas Jinja	Componentes de React (JSX)
`request.form` en un POST	Server Action que recibe los datos del formulario
SQLite o listas en memoria	PostgreSQL en Supabase
`login_required` o revisar permisos en la vista	Protección en el servidor + políticas RLS en la base de datos
`flask run`	`npm run dev`

Pregunta pendiente: ¿tu app de tareas guardaba los datos en una base de datos (por ejemplo SQLite) o en memoria? Si usaste SQL, díselo a Diego: cambia tu punto de partida en la ruta de aprendizaje.

Tu calendario por sprint

Sprint	Qué entregas
0 · 28 oct – 8 nov	Preparar tu computador; primer PR real; trabajar en pareja con Daniel durante M0; aprender JS moderno, React y App Router; leer completo el documento de contexto.
1 · 9 – 15 nov	Tabla requests + RLS (mié 11); formulario de nueva solicitud (dom 15).
2 · 16 – 22 nov	Listado por rol, detalle, editar y cancelar; M3 terminado (dom 22).
3 · 23 – 29 nov	Tabla messages + RLS; enviar y ver una conversación (dom 29).
4 · 30 nov – 6 dic	Lista de conversaciones, leídos y botones desde las cotizaciones; M5 terminado (dom 6).
5 · 7 – 11 dic	Estabilización y presentación.

Ruta de aprendizaje (justo a tiempo)

Aprende cada tema justo antes de la tarea que lo necesita, no todo al principio. El orden importa: cada punto se apoya en el anterior.

Semana 1 (Sprint 0)

1. Git en equipo: clonar, rama, commits pequeños, push, pull request y actualizar tu rama con main (Anexo C).
2. JavaScript moderno comparado con Python: const y let; arreglos con map y filter (como las list comprehensions); objetos (como los diccionarios); async/await.
3. React: componentes, props y JSX.
4. Next.js App Router: carpetas como rutas, `page.js`, `layout.js`, componentes de servidor y de cliente.

Antes de M3 (Sprint 1)

1. Formularios con Server Actions (el equivalente a un POST en Flask) y validación en el servidor.
2. SQL en PostgreSQL: crear tablas, claves foráneas, CHECK, consultas con filtro y orden.
3. Supabase desde el servidor, con el cliente de Diego.
4. RLS: políticas para requests (pide 1 hora con Diego antes de escribirlas).
5. Rutas dinámicas (`[requestId]`).

Antes de M5 (Sprint 3)

1. Políticas RLS que consultan otras tablas (requests y quotes).
2. Agrupar datos para armar la lista de conversaciones.
3. (Opcional) Supabase Realtime.

Módulo 3 — Publicación y gestión de solicitudes

Objetivo	El cliente publica solicitudes con lo que necesita, la cantidad, la ubicación, la fecha límite y la categoría; las administra (editar y cancelar), y los proveedores consultan las solicitudes abiertas.
Alcance	Mínimo = completo. Todo M3 es obligatorio y está en la ruta crítica: M4 se construye sobre tus solicitudes.
Fuera de alcance	Fotos adjuntas (solo si sobra tiempo), solicitudes privadas a un solo proveedor y búsqueda por texto.
Depende de	Diego (M1 y funciones de ayuda, sáb 7 nov); Daniel (categories, mar 10 nov).
Desbloquea a	Diego (M4: las cotizaciones se hacen sobre una solicitud).

Tabla `requests`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
client_id	uuid → profiles.id	Obligatorio: el cliente que la publica.
category_id	uuid → categories.id	Obligatorio.
title	text	Obligatorio, 5 a 120 caracteres.
description	text	Obligatorio, hasta 2000 caracteres.
quantity	integer	Mayor que 0.
location	text	Obligatorio: ciudad y barrio.
deadline	date	Al crear, debe ser una fecha futura (se valida en el servidor).
status	text	open, awarded o cancelled; por defecto open. awarded solo lo pone la función de aceptar de Diego.
is_hidden	boolean	Falso por defecto; solo el admin lo cambia.
created_at	timestamptz	Automático.

Políticas RLS de `requests`

Operación	Quién	Condición
SELECT	Cliente, proveedor, admin	El cliente ve todas las suyas; los proveedores ven las open no ocultas y, cuando exista quotes, también las que cotizaron; el admin ve todas.
INSERT	Cliente	Rol client; `client_id` es el usuario actual; status open.
UPDATE	Cliente dueño	Solo si está open: editar campos o cancelar. No puede cambiar `client_id` ni `is_hidden`, ni poner awarded.
DELETE	Nadie	Se cancela en vez de borrar: conserva el historial y no rompe las cotizaciones [SUPUESTO].

Regla de edición [SUPUESTO, decide el PO]: una solicitud solo se edita mientras no tenga cotizaciones, para que nadie cambie las condiciones después de que los proveedores cotizaron.

RLS controla filas, no columnas. Para que el dueño no cambie `client_id`, `is_hidden` ni el estado a awarded, el servidor no envía esos campos y la base lo bloquea con el mecanismo que Diego usa para el rol en profiles. Antes de escribir tus políticas, pide 1 hora con Diego.

Rutas y archivos

- `src/app/solicitudes/nueva/page.js` — formulario (solo clientes).
- `src/app/solicitudes/page.js` — cliente: «Mis solicitudes» con su estado; proveedor: «Solicitudes abiertas» con filtro por categoría (reutiliza CategoryFilter de Daniel).
- `src/app/solicitudes/[requestId]/page.js` — detalle con acciones según el rol: el dueño edita, cancela y va a comparar cotizaciones (página de Diego); el proveedor ve el formulario de cotización (componente de Diego).
- `src/app/solicitudes/[requestId]/editar/page.js` [SUPUESTO: ruta nueva] — edición.
- `src/components/requests/` — RequestForm, RequestCard y RequestStatusBadge.
- `src/services/requests.js` — todas las consultas de solicitudes.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Primer PR de práctica: agrega tu fila a la tabla de integrantes del documento de contexto en `docs/`.	mar 3 nov	Tu PR fue revisado y fusionado sin conflictos.
2	Tabla requests + RLS en dev.	mié 11 nov	Un cliente crea; un proveedor no puede crear; el cliente B no ve las solicitudes canceladas de A.
3	Formulario de nueva solicitud con Server Action y validaciones en el servidor: fecha futura, cantidad mayor que 0 y categoría existente.	dom 15 nov	Los datos inválidos muestran el error junto al campo y no se guardan.
4	Listado según el rol + filtro por categoría.	mar 17 nov	Cliente y proveedor ven listas distintas.
5	Detalle; 404 si no existe o no tiene permiso.	jue 19 nov	Un id ajeno o inventado muestra la página 404.
6	Editar (si está open y sin cotizaciones) y cancelar.	sáb 21 nov	Una solicitud cancelada desaparece para los proveedores.
7	Con Diego: colocar QuoteForm en el detalle y ajustar el SELECT para los proveedores que cotizaron.	sáb 21 nov	Un proveedor sigue viendo una solicitud adjudicada que cotizó.
8	Prueba cruzada (la hace Daniel) y publicación con solicitudes reales del piloto.	dom 22 nov	Checklist de la Definition of Done completo (Anexo G).

Criterios de aceptación

- **Dado** un cliente con sesión, **cuando** publica una solicitud con datos válidos, **entonces** aparece en «Mis solicitudes» como abierta y los proveedores la ven en «Solicitudes abiertas».
- **Dado** un cliente, **cuando** pone una fecha límite pasada o una cantidad 0, **entonces** el servidor rechaza la solicitud y muestra el error junto al campo.
- **Dado** un proveedor, **cuando** intenta crear una solicitud desde la app o directo contra Supabase, **entonces** la operación falla.
- **Dado** el cliente A, **cuando** intenta editar o cancelar la solicitud del cliente B cambiando el id en la URL, **entonces** la operación falla.
- **Dado** un cliente, **cuando** cancela una solicitud abierta, **entonces** deja de aparecer para los proveedores y no recibe más cotizaciones.

Pruebas de permisos

Prueba como visitante, cliente A, cliente B, proveedor y admin: crear, ver la lista, ver el detalle, editar y cancelar. Anota el resultado esperado y el obtenido.

Errores típicos y cómo evitarlos

- Poner «use client» en toda la página: solo lo necesita el componente interactivo.
- Fechas con hora y zona horaria: guarda solo la fecha.
- La lista no se actualiza después de crear: hay que revalidar la ruta.
- Proteger ocultando botones en vez de verificar en el servidor y en RLS.
- Pegar código de la IA escrito para Flask, para el router viejo de Next.js (carpeta `pages/`) o para TypeScript.

Coordinación con el equipo

- Daniel: necesitas categories el mar 10 nov y su componente CategoryFilter.
- Diego: acuerden dónde va QuoteForm y el ajuste del SELECT para proveedores que cotizaron.
- Iker: coloca su ReportButton en el detalle de la solicitud y, en el Sprint 4, su ReviewForm cuando la solicitud esté adjudicada.

Módulo 5 — Mensajería y comunicación entre usuarios

Objetivo	Mensajes de texto entre el cliente de una solicitud y cada proveedor que la cotizó, guardados en PostgreSQL.
Alcance	**Mínimo (compromiso):** enviar, ver la conversación al recargar, lista de conversaciones y marcar como leídos. **Completo** (solo si sobra tiempo y el PO lo aprueba): Supabase Realtime para ver mensajes nuevos sin recargar y contador de no leídos en el encabezado.
Fuera de alcance	Archivos o imágenes, chat sin solicitud, chats de grupo y avisos por correo.
Depende de	M3 (tuyo); quotes de Diego (mar 17 nov); vista public_profiles de Diego para mostrar nombres.
Desbloquea a	Nadie: es un módulo final.

Quién puede hablar [SUPUESTO, confirmar con el PO]: solo el cliente dueño de la solicitud y el dueño de un negocio que le envió una cotización. Evita mensajes no deseados y simplifica los permisos.

Tabla `messages`

Campo	Tipo sugerido	Reglas
id	uuid	Clave primaria, automática.
sender_id	uuid → profiles.id	Quien envía; debe ser el usuario actual.
receiver_id	uuid → profiles.id	Quien recibe; distinto de sender_id (CHECK).
request_id	uuid → requests.id	La solicitud de la conversación.
content	text	Obligatorio, 1 a 2000 caracteres.
created_at	timestamptz	Automático.
read_at	timestamptz	Vacío = no leído.

Políticas RLS de `messages`

Operación	Quién	Condición
SELECT	Participantes	Solo si eres `sender_id` o `receiver_id`. El admin no lee mensajes privados [SUPUESTO].
INSERT	Participantes válidos	`sender_id` es el usuario actual y la pareja cumple la regla (cliente de la solicitud ↔ dueño de un negocio que cotizó); la solicitud no está oculta.
UPDATE	Destinatario	Solo marcar `read_at`; nadie edita el contenido.
DELETE	Nadie	—

Para la regla de INSERT, crea una función de ayuda en la base (por ejemplo `can_message`) que reciba la solicitud y la otra persona. Revísala con Diego antes de fusionar.

Rutas y archivos

- `src/app/mensajes/page.js` — lista de conversaciones (solicitud + otra persona, último mensaje, no leídos); al elegir una se muestra la conversación y el formulario de envío.
- Botón «Enviar mensaje» en la comparación de cotizaciones (cliente → proveedor) y en «Mis cotizaciones» (proveedor → cliente): coordinar con Diego.
- `src/components/messages/` — ConversationList, MessageThread y MessageForm (carpeta nueva respecto al documento).

- `src/services/messages.js` — consultas y envío.

Pasos en orden (cada paso es un pull request pequeño)

#	Paso	Fecha meta	Cómo verificar que quedó bien
1	Tabla messages + RLS + función de ayuda en dev.	lun 23 nov	Un tercero no puede leer ni escribir en una conversación ajena.
2	Enviar y ver una conversación.	jue 26 nov	El mensaje aparece al recargar, con el nombre de la persona (sin correo).
3	Lista de conversaciones y marcar como leídos.	dom 29 nov	Al abrir una conversación, sus mensajes quedan leídos.
4	Botones de entrada desde las cotizaciones (con Diego).	mié 2 dic	Desde una cotización se abre la conversación correcta.
5	Prueba cruzada (la hace Iker) y publicación con conversaciones reales del piloto.	dom 6 dic	Checklist de la Definition of Done completo (Anexo G).
6	(Opcional, solo con aprobación del PO) Realtime.	—	Un mensaje nuevo aparece sin recargar.

Criterios de aceptación

- **Dado** un cliente con una solicitud cotizada por el proveedor A, **cuando** le escribe a A, **entonces** A ve el mensaje en /mensajes con el nombre del cliente y la solicitud.
- **Dado** un proveedor C que no cotizó esa solicitud, **cuando** intenta escribirle al cliente, **entonces** la operación falla.
- **Dado** un usuario que no participa en una conversación, **cuando** intenta leer sus mensajes, **entonces** no obtiene nada.
- **Dado** el destinatario, **cuando** abre la conversación, **entonces** los mensajes quedan leídos; el remitente no puede editar los mensajes enviados.

Errores típicos y cómo evitarlos

- Traer todos los mensajes y filtrarlos en el navegador: es una fuga de datos.
- Mostrar correos: usa la vista public_profiles.
- Pedir el nombre de cada mensaje con una consulta distinta: trae los nombres en una sola consulta.
- Activar Realtime sin políticas RLS correctas.

Coordinación con el equipo

- Diego: quotes debe existir en dev el mar 17 nov; acuerden los botones de entrada desde las cotizaciones.
- Iker: hace la prueba cruzada de M5 intentando leer y escribir en conversaciones ajenas.

Anexo — Acuerdos comunes del equipo

Este anexo es igual en las cuatro guías. Si cambia, se cambia en todas.

A. Calendario del equipo

Sprint	Fechas	Meta del equipo
0	mié 28 oct – dom 8 nov	Proyecto publicado (M0), entornos listos, todos con su computador preparado, M1 en marcha
1	lun 9 – dom 15 nov	M1 terminado; tablas categories, businesses, requests y reports creadas con RLS
2	lun 16 – dom 22 nov	M2 y M3 terminados; envío de cotizaciones; panel de admin básico
3	lun 23 – dom 29 nov	M4 terminado: solicitud → cotización → elegir, demostrable con usuarios reales
4	lun 30 nov – dom 6 dic	M5 y M6 (mínimos) terminados; M7 mínimo terminado
5	lun 7 – vie 11 dic	Estabilización: pruebas, errores y presentación. Entrega: vie 11 dic

Festivos en Colombia dentro del periodo: lun 2 nov, lun 16 nov y mar 8 dic.

B. Preparar tu computador (Windows, PowerShell)

#	Paso	Comando o acción	Cómo verificar
1	Instalar Node.js LTS	`winget install OpenJS.NodeJS.LTS` (o el instalador LTS de nodejs.org)	`node -v` y `npm -v` muestran versión
2	Instalar Git	`winget install Git.Git`	`git --version`
3	Permitir scripts de npm	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` (responde S)	`npx -v` funciona sin el error «scripts deshabilitados»

4	Configurar Git	`git config --global user.name "Tu Nombre"`
		`git config --global user.email "correo de tu GitHub"`
		`git config --global init.defaultBranch main`
			`git config --global --list`

5	Extensiones de VS Code	ESLint, Tailwind CSS IntelliSense y Prettier; opcional: GitLens	Aparecen en la pestaña Extensiones

6	Clonar fuera de OneDrive	`mkdir C:\dev`
		`cd C:\dev`
		`git clone https://github.com/diegodani2030/App-Offero.git`
			Existe `C:\dev\App-Offero`

7	Instalar dependencias (cuando M0 esté en main)	`cd C:\dev\App-Offero` `npm install`	Termina sin errores en rojo
8	Variables de entorno	`Copy-Item .env.example .env.local` y llena los valores que Diego te envía por mensaje privado	`.env.local` existe y `git status` no lo muestra
9	Correr la app	`npm run dev` y abre http://localhost:3000	Ves la portada de Offero

Abre una PowerShell nueva después de instalar Node y Git. El correo de Git debe ser el de tu cuenta de GitHub. La primera vez que hagas push, Windows abrirá el navegador para iniciar sesión en GitHub.

C. Flujo de Git paso a paso

#	Qué haces	Comando o acción
1	Toma una tarea del tablero y muévela a «En curso»	GitHub Projects
2	Actualiza main	`git switch main` → `git pull`
3	Crea tu rama: tipo/mN-descripcion (tipos: feat, fix, chore, docs)	`git switch -c feat/m3-formulario-solicitud`
4	Guarda avances pequeños con mensajes en español: tipo(módulo): qué hace	`git add .` → `git commit -m "feat(m3): agrega formulario de nueva solicitud"`
5	Sube la rama	`git push -u origin feat/m3-formulario-solicitud` (después, solo `git push`)
6	Abre el pull request hacia main con la plantilla: qué cambia, cómo probarlo, capturas y checklist. Escribe «Cierra #N» para enlazar la tarea	GitHub
7	Revisión: al menos un compañero. Si toca base de datos, autenticación o archivos compartidos, revisa Diego. Corrige con nuevos commits en la misma rama	GitHub
8	Fusión: la hace Diego con «Create a merge commit» (por la restricción de Vercel Hobby); luego se borra la rama	GitHub
9	Actualiza tu rama con main cada día	`git switch main` → `git pull` → `git switch tu-rama` → `git merge main`
10	Si hay conflicto, VS Code muestra las opciones (cambio actual, entrante o ambos). Si no entiendes el otro cambio, pregúntale a su autor. Luego guarda y haz commit	`git add .` → `git commit`

- Nunca hagas commits directos a main ni `git push --force`.
- Pull requests de menos de un día de trabajo: son más fáciles de revisar y casi no generan conflictos.
- No subas `.env.local`, `node_modules` ni claves. Si `git status` los muestra, detente y pregunta.

D. Convenciones de base de datos

- **Dos proyectos de Supabase** [SUPUESTO; se confirma en la Parte 3]: `offero-dev` para desarrollo y pruebas (ahí sí puedes crear datos de prueba) y `offero-prod` solo con datos reales (lo usa la versión de producción en Vercel). Tu `.env.local` y las previews apuntan a dev.
- **Cada cambio de esquema es un archivo** en `supabase/sql/` con nombre `AAAAMMDD_HHMM_descripcion.sql` (por ejemplo `20261111_1530_requests.sql`). La fecha y hora evitan choques entre cuatro personas. Se prueba en dev, va en el PR y Diego lo aplica en prod al fusionar. Nunca edites un archivo ya aplicado: crea uno nuevo. Nadie cambia tablas desde el panel sin dejar su archivo.
- **Todas las tablas**: id uuid automático (profiles usa el id de Auth), created_at automático, claves foráneas explícitas y restricciones CHECK para estados y números.
- **RLS se activa en el mismo archivo que crea la tabla.** Una tabla sin políticas no se fusiona.
- **Contenido moderable** (businesses, requests, reviews) incluye `is_hidden` desde su creación, para que M7 no tenga que modificar tablas ajenas [SUPUESTO; registrar en la sección 11 del documento].
- **Funciones de ayuda de Diego**: `is_admin()` y `get_my_role()`. Úsenlas en las políticas en vez de repetir consultas.

- **Nombres de personas**: siempre desde la vista `public_profiles`. Los correos nunca se muestran a otros usuarios.
- **Validación en tres capas**: formulario (comodidad), servidor de Next.js (seguridad) y base de datos (restricciones y RLS, la última barrera).

Tabla.campo	Valores acordados (fuente única)
`profiles.role`	client, provider, admin
`requests.status`	open (recibe cotizaciones), awarded (adjudicada), cancelled
`quotes.status`	pending, accepted, rejected, withdrawn
`reports.status`	open, reviewed, dismissed
`reports.target_type`	business, request, review

E. Archivos compartidos

Archivo o carpeta	Dueño	Regla
`src/app/layout.js`, `globals.css`, `src/components/layout/`	Daniel	Avisa en Discord antes de cambiarlos; cambios pequeños
`src/lib/supabase/` y archivo de sesión (proxy o middleware)	Diego	Nadie crea otro cliente de Supabase
`supabase/sql/`	Dueño de cada tabla; aprueba Diego	No editar archivos ya aplicados
`package.json` y `package-lock.json`	Todos, con acuerdo	No instales paquetes sin acordarlo (restricción de stack). Si hay conflicto en el lock: acepta la versión de main y vuelve a ejecutar `npm install`
`docs/` (documento de contexto)	Cada uno, su módulo	Actualiza las secciones 8, 9 y 10 al avanzar
`src/components/ui/`	Quien crea el componente	Si cambias un componente compartido (ReportButton, CategoryFilter), avisa

F. Comunicación, regla de atasco e IA

- **Canales** [SUPUESTO]: Discord para el trabajo (#daily para el avance diario, #ayuda para bloqueos, #prs para pedir revisión). WhatsApp solo para urgencias.
- **Regla de atasco**: después de 45 minutos intentando solo, escribe en #ayuda qué intentabas, el archivo, el mensaje de error completo y qué probaste. Si en 2 horas nadie responde, menciona a Diego. Un bloqueo de más de un día se dice en la Daily.
- **Revisiones**: responde las que te pidan en menos de 24 horas.
- **IA**: pega el documento de contexto al iniciar cada conversación; pide explicaciones, no solo código; verifica que use JavaScript, App Router y `@supabase/ssr` (no TypeScript, ni la carpeta `pages/`, ni Flask); no subas nada que no puedas explicar en la revisión; nunca pegues claves en la IA.

G. Definition of Done

Una historia está hecha cuando:

- cumple sus criterios de aceptación;
- su pull request fue revisado y fusionado;
- funciona en la Preview y luego en producción;
- sus permisos se probaron con dos cuentas de roles distintos;

- no deja errores en la consola del navegador;
- el documento de contexto refleja cualquier cambio.

Un módulo está Terminado cuando (regla del documento de contexto):

- funciona en la versión publicada en Vercel;
- sus operaciones principales están probadas por otra persona (prueba cruzada);
- no depende de datos ficticios en producción;
- la sección 8 del documento dice Terminado y Publicado: Sí.